CLOUD COMPUTING AND DEVOPS ENGINEERING · WEEK 8 · MIDTERM PROJECT

Message Queue + Visit Counter with Redis

Instructor: M.Sc. Abdelhakim Rashid

Group Members	Ahmad Ibrahim Elmufti – **4866** Aya Khaled Abuoud – **4834** Seraj Nouri AlFaitoury – **5102**
Date	May 10, 2026
GitHub	github.com/ahm4dmufti/Cloude_Computing_Midterm

---

1. Project Title and Description

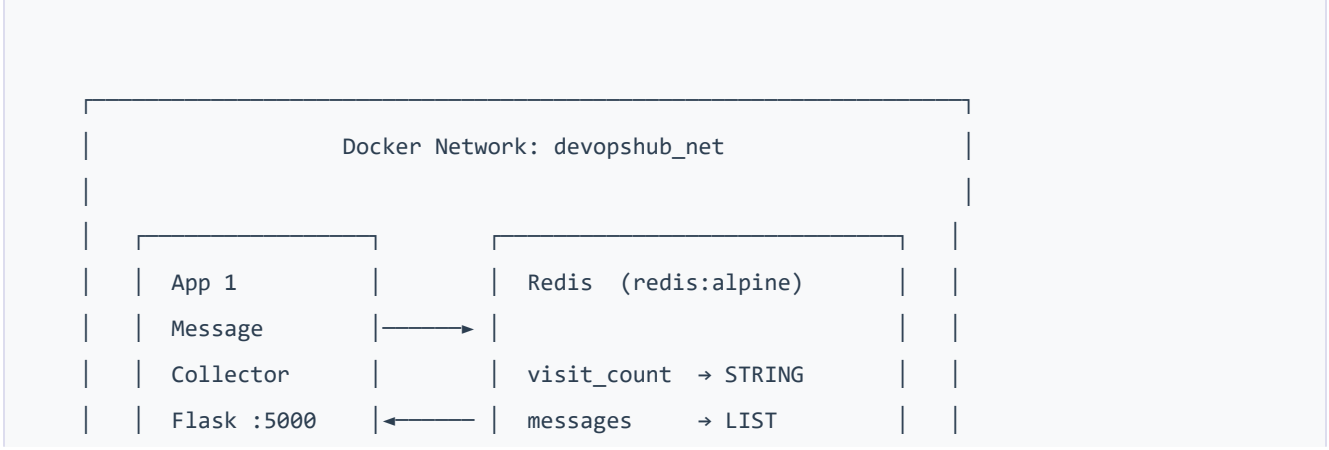
Title: DevOpsHub – Message Queue + Visit Counter with Redis

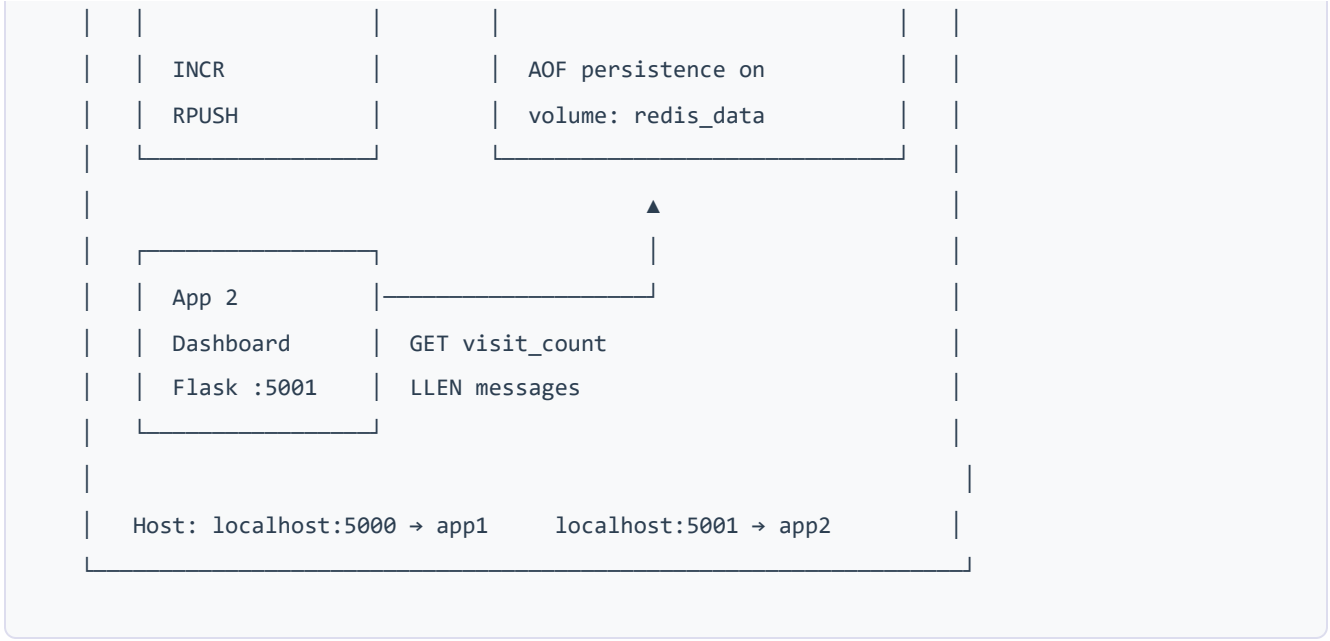
We built a feedback system made up of two Flask apps and a Redis container, all wired together with Docker Compose. App 1 is a form where users can type and submit a message. App 2 is a dashboard that shows how many messages were submitted and how many times the form page was visited.

The two apps don't talk to each other directly — they both connect to the same Redis instance. App 1 writes to it, App 2 reads from it. All three containers are on the same Docker network, so they reach each other by name instead of IP.

2. Architecture

Below is a diagram showing how the three containers are connected. App 1 writes to Redis, App 2 reads from Redis, and neither app talks directly to the other.





Service	Image	Port	Role
redis	redis:alpine	6379 (internal only)	Shared database
app1	project-app1	5000 → 5000	Collects messages, counts visits
app2	project-app2	5001 → 5001	Shows counts from Redis

3. How the Two Apps Interact with Redis

Both apps connect to Redis with `host='redis'`. Because all three containers are on the same Docker network, Docker resolves the name `redis` to the right container — no IP needed.

What App 1 does

On every page load it runs `INCR visit_count`, which bumps the counter by 1 and returns the new value to display. When someone submits the form, it runs `RPUSH messages <text>` to append the message to a Redis list, then redirects back.

What App 2 does

App 2 only ever reads. On each load it calls `LLEN messages` to get the list length and `GET visit_count` to get the counter, then shows both on the page. It never writes to Redis.

Redis Key	Type	Who writes	Who reads	Command
`visit_count`	String	App 1	App 2	INCR / GET
`messages`	List	App 1	App 2	RPUSH / LLEN

4. Prerequisites

To run this project you only need:

- **Docker Desktop** installed and running (it includes Docker Compose v2)
- Internet connection the first time, to pull the base images from Docker Hub
- Ports **5000** and **5001** free on your machine

No Python or Redis installation is needed on the host — everything runs inside the containers.

5. Step-by-Step Solution

Step 1 – Create the folder structure

Open a terminal and create the project folders:

```
mkdir project
cd project
mkdir app1
mkdir app2
```

The structure we are going to build:

```
project/
├── app1/
│   ├── app.py
│   ├── requirements.txt
│   └── Dockerfile
├── app2/
│   ├── app.py
│   ├── requirements.txt
│   └── Dockerfile
└── docker-compose.yml
```

Step 2 – Write App 1 (Message Collector)

Create `app1/requirements.txt`:

```
flask==3.1.0
redis==5.2.1
```

Create `app1/app.py`:

```
from flask import Flask, request, redirect, url_for
import redis

app = Flask(__name__)
r = redis.Redis(host='redis', port=6379, decode_responses=True)

HTML = """
<!DOCTYPE html>
<html>
<head>
  <title>Message Collector</title>
  <style>
    body {{ font-family: Arial, sans-serif; max-width: 600px; margin: 60px auto; backgrou
    .card {{ background: white; padding: 32px; border-radius: 10px; box-shadow: 0 2px 8p
    h1 {{ color: #2c3e50; margin-bottom: 4px; }}
    .badge {{ display: inline-block; background: #3498db; color: white; padding: 4px 12p
              border-radius: 20px; font-size: 14px; margin-bottom: 24px; }}
    textarea {{ width: 100%; padding: 10px; border: 1px solid #ccc; border-radius: 6px;
                font-size: 15px; box-sizing: border-box; }}
    button {{ margin-top: 12px; padding: 10px 28px; background: #2ecc71; color: white;
              border: none; border-radius: 6px; font-size: 15px; cursor: pointer; }}
  </style>
</head>
<body>
  <div class="card">
    <h1>DevOpsHub Feedback</h1>
    <span class="badge">Page visits: {visits}</span>
    <form method="POST" action="/submit">
      <textarea name="message" rows="4"
        placeholder="Enter your message, feedback or suggestion..." required></textarea>
      <br>
      <button type="submit">Submit Message</button>
    </form>
  </div>
</body>
</html>
"""

@app.route('/', methods=['GET'])
def index():
    visits = r.incr('visit_count')
    return HTML.format(visits=visits)
```

```
@app.route('/submit', methods=['POST'])
def submit():
    message = request.form.get('message', '').strip()
    if message:
        r.rpush('messages', message)
    return redirect(url_for('index'))

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

Step 3 – Write App 2 (Dashboard)

Create `app2/requirements.txt`:

```
flask
redis
```

Create `app2/app.py`:

```
from flask import Flask
import redis

app = Flask(__name__)
r = redis.Redis(host='redis', port=6379, decode_responses=True)

HTML = """
<!DOCTYPE html>
<html>
<head>
  <title>DevOpsHub Dashboard</title>
  <style>
    body {{ font-family: Arial, sans-serif; max-width: 600px; margin: 60px auto; backgrou
    .card {{ background: white; padding: 32px; border-radius: 10px; box-shadow: 0 2px 8p
    h1 {{ color: #2c3e50; margin-bottom: 24px; }}
    .stat {{ display: flex; justify-content: space-between; align-items: center;
             padding: 16px 0; border-bottom: 1px solid #eee; }}
    .stat:last-child {{ border-bottom: none; }}
    .label {{ font-size: 16px; color: #555; }}
    .value {{ font-size: 28px; font-weight: bold; color: #3498db; }}
    .footer {{ margin-top: 16px; font-size: 12px; color: #aaa; text-align: right; }}
  </style>
</head>
```

```
<body>
  <div class="card">
    <h1>Dashboard</h1>
    <div class="stat">
      <span class="label">Total Messages Collected</span>
      <span class="value">{messages}</span>
    </div>
    <div class="stat">
      <span class="label">Total Page Visits (App 1)</span>
      <span class="value">{visits}</span>
    </div>
    <p class="footer">Data refreshes on every page load</p>
  </div>
</body>
</html>
"""

@app.route('/')
def dashboard():
    messages = r.llen('messages')
    visits = r.get('visit_count') or 0
    return HTML.format(messages=messages, visits=visits)

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5001)
```

Step 4 – Write the Dockerfile for App 1

Create `app1/Dockerfile`:

```
FROM python:3.11-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY app.py .

EXPOSE 5000
```

```
CMD ["python", "app.py"]
```

Step 5 – Write the Dockerfile for App 2

Create `app2/Dockerfile`. It is the same as App 1 except the port:

```
FROM python:3.11-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY app.py .

EXPOSE 5001

CMD ["python", "app.py"]
```

Step 6 – Write docker-compose.yml

Create `docker-compose.yml` in the root of the project:

```
services:
  redis:
    image: redis:alpine
    container_name: redis
    volumes:
      - redis_data:/data
    command: redis-server --appendonly yes
    networks:
      - devopshub_net
    healthcheck:
      test: ["CMD", "redis-cli", "ping"]
      interval: 5s
      timeout: 3s
      retries: 5

  app1:
    build: ./app1
```

```
    container_name: app1
    ports:
      - "5000:5000"
    depends_on:
      redis:
        condition: service_healthy
    restart: unless-stopped
    networks:
      - devopshub_net

  app2:
    build: ./app2
    container_name: app2
    ports:
      - "5001:5001"
    depends_on:
      redis:
        condition: service_healthy
    restart: unless-stopped
    networks:
      - devopshub_net

volumes:
  redis_data:

networks:
  devopshub_net:
    driver: bridge
```

The healthcheck pings Redis every 5 seconds, and `depends_on` with `condition: service_healthy` makes the apps wait until Redis is actually ready — not just started. `restart: unless-stopped` means if an app crashes it comes back on its own. The named volume keeps the data if the containers are stopped.

Step 7 – Build and run everything

From the `project/` folder run:

```
docker compose up --build -d
```

This builds the two app images, pulls Redis, creates the network and volume, and starts all three containers in the background. The first run takes a few minutes because it downloads the base

images.

Step 8 – Check that all containers are up

```
docker compose ps
```

You should see all three containers with status **Up**:

```
NAME    IMAGE          COMMAND          SERVICE  CREATED         STATUS         PORTS
app1    project-app1   "python app.py"  app1     20 seconds ago  Up 19 seconds  0.0.0.0:5
app2    project-app2   "python app.py"  app2     20 seconds ago  Up 19 seconds  0.0.0.0:5
redis   redis:alpine   "docker-entry…"  redis    21 seconds ago  Up 19 seconds  6379/tcp
```

Step 9 – Test the application

1. Open **http://localhost:5000** — the visit counter goes up by 1 on each load.
2. Type something in the form and click Submit Message.
3. Open **http://localhost:5001** — you will see the message count and visit count updated.

6. docker-compose.yml

```
services:
  redis:
    image: redis:alpine
    container_name: redis
    volumes:
      - redis_data:/data
    command: redis-server --appendonly yes
    networks:
      - devopshub_net
    healthcheck:
      test: ["CMD", "redis-cli", "ping"]
      interval: 5s
      timeout: 3s
      retries: 5

  app1:
```

```
    build: ./app1
    container_name: app1
    ports:
      - "5000:5000"
    depends_on:
      redis:
        condition: service_healthy
    restart: unless-stopped
    networks:
      - devopshub_net

  app2:
    build: ./app2
    container_name: app2
    ports:
      - "5001:5001"
    depends_on:
      redis:
        condition: service_healthy
    restart: unless-stopped
    networks:
      - devopshub_net

volumes:
  redis_data:

networks:
  devopshub_net:
    driver: bridge
```

7. Dockerfile – Web App 1

```
FROM python:3.11-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY app.py .

EXPOSE 5000
```

```
CMD ["python", "app.py"]
```

`python:3.11-slim` keeps the image small. We copy `requirements.txt` before the app code so Docker caches the pip install — changing `app.py` later won't trigger a full reinstall.

8. Dockerfile – Web App 2

```
FROM python:3.11-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY app.py .

EXPOSE 5001

CMD ["python", "app.py"]
```

Same as App 1's Dockerfile, just port 5001 instead of 5000.

9. Docker Images

After building, running `docker images` shows these images were created:

Repository	Tag	Image ID	Size
project-app1	latest	2943b5895b1e	216 MB
project-app2	latest	19cf64f0f072	216 MB
redis	alpine	d146f83b1e0f	134 MB

The app images are ~216 MB each — that's `python:3.11-slim` plus Flask and the Redis client. Redis on Alpine is much lighter at 134 MB.

10. Access Instructions

Once the containers are running, open a browser and go to:

App	URL	What you will see
App 1 – Message Collector	`http://localhost:5000`	A form to submit messages with a visit counter at the top
App 2 – Dashboard	`http://localhost:5001`	Total messages collected and total page visits

To stop the containers:

```
docker compose down
```

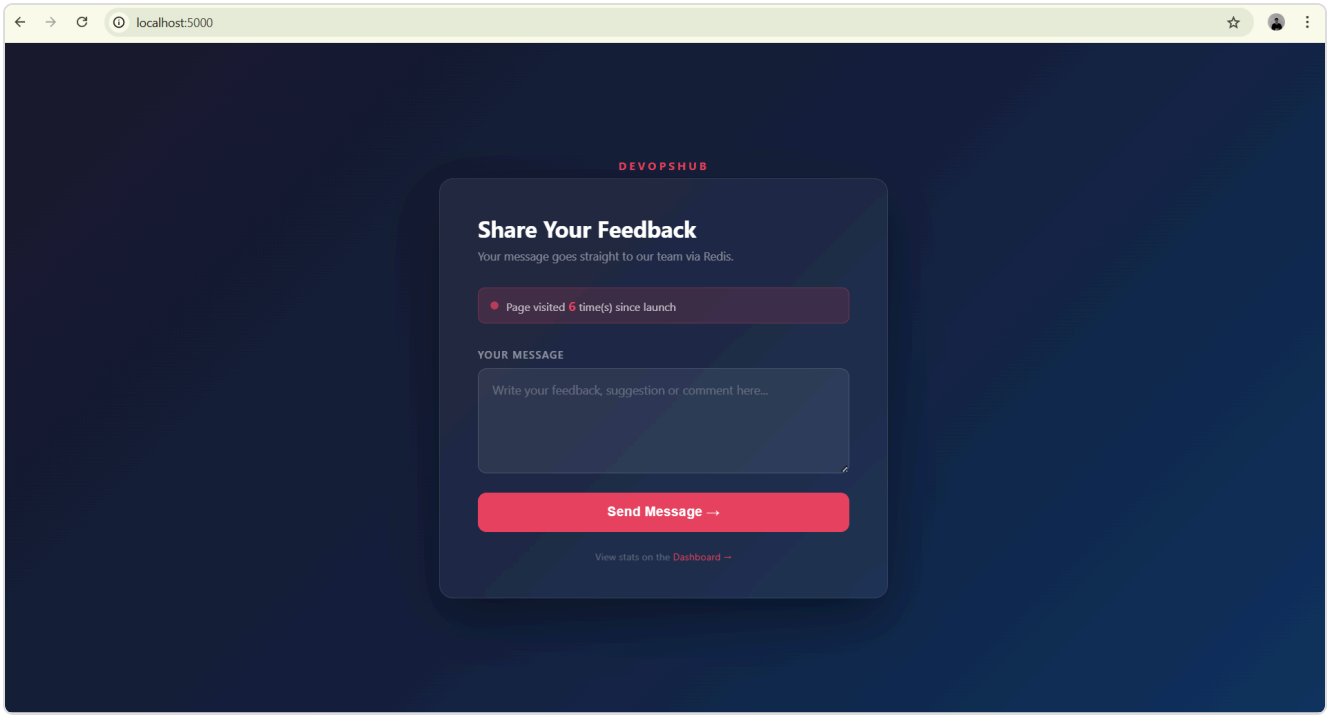
11. Screenshots

Screenshot 1 – docker compose ps showing all three services Up

```
PS C:\Users\ahmad\Desktop\project> docker compose ps
NAME      IMAGE          COMMAND                  SERVICE   CREATED          STATUS          PORTS
app1      project-app1   "python app.py"          app1      18 minutes ago   Up 18 minutes   0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp
app2      project-app2   "python app.py"          app2      18 minutes ago   Up 18 minutes   0.0.0.0:5001->5001/tcp, [::]:5001->5001/tcp
redis     redis:alpine   "docker-entrypoint.s…"   redis     18 minutes ago   Up 18 minutes   6379/tcp
PS C:\Users\ahmad\Desktop\project>
```

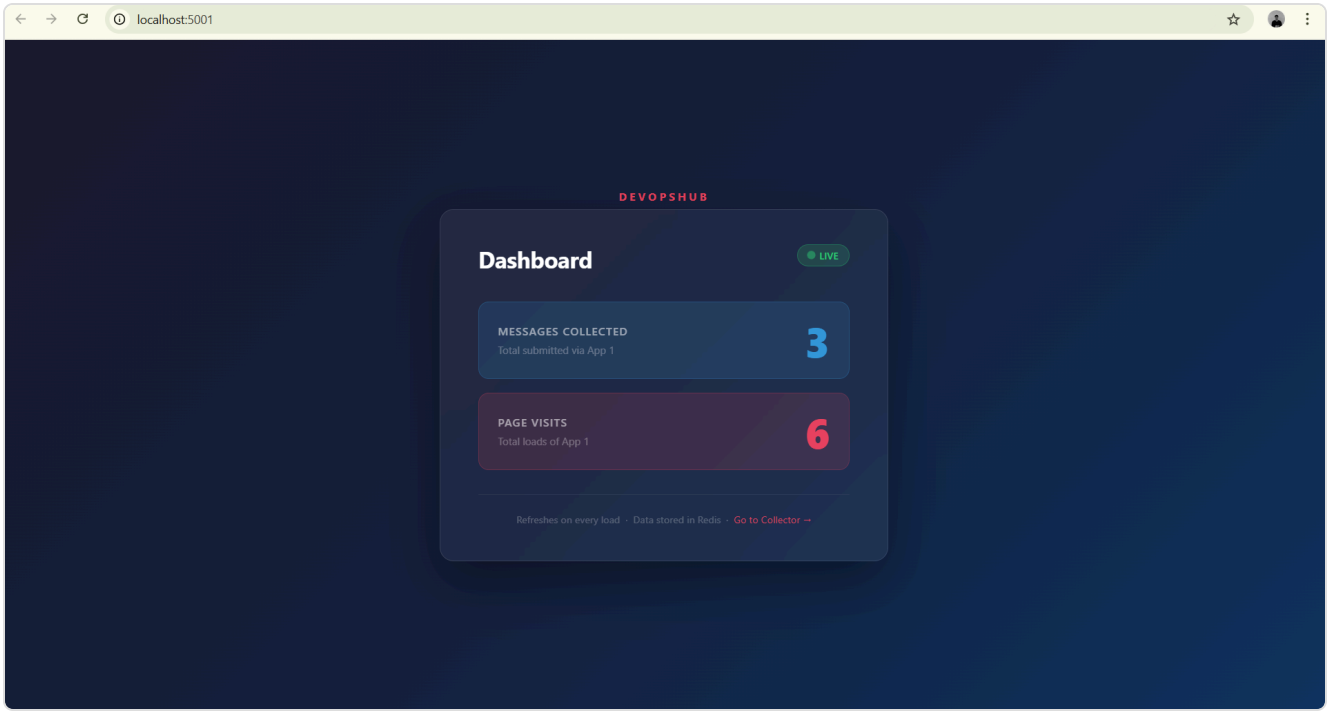
All three containers running: app1 (port 5000), app2 (port 5001), redis (port 6379)

Screenshot 2 – Browser showing App 1 (Message Collector) at localhost:5000



App 1 running at http://localhost:5000 — feedback form with visit counter badge

Screenshot 3 – Browser showing App 2 (Dashboard) at localhost:5001



App 2 running at http://localhost:5001 — total messages and visit count pulled from Redis